

第十四周作业报告

项目	内容
课程名称	ROS 机器人编程实践
作业周次	第 14 周
平台	ROS + Turtlesim
组别	王昕昊组
成员	王昕昊
日期	2026 年 6 月 4 日

一、项目概述

本次作业基于 ROS2 架构与 Turtlesim 仿真环境，设计并实现了一套完整的“ROS2 TurtleSim 迷宫控制台”系统。该项目旨在探索机器人系统在“感知-决策-控制”闭环中的工程实现，通过构建 Web 前端与 ROS2 后端节点的通信链路，实现了以下核心功能：

- 1. **实时状态监控**：在浏览器端低延迟显示机器人的位姿、速度及运动状态。
- 2. **双模控制策略**：支持用户通过键盘/按钮进行手动遥控，或切换至基于状态机的自动避障探索模式。
- 3. **可视化交互**：利用 HTML5 Canvas 技术，将抽象的 ROS 话题数据转化为直观的迷宫地图与探索轨迹。

该系统不仅验证了 ROS2 节点通信机制，还实践了跨语言（Python/JavaScript）、跨协议（ROS2 DDS/WebSocket）的异构系统集成方案。

二、HTML 控制台界面与交互逻辑

前端控制台（index.html）采用现代化的左右分栏响应式布局，通过 WebSocket 协议与后端保持全双工通信，主要包含以下功能模块：

- 1. **可视化控制面板**：
 - 提供方向控制按钮组（前进、后退、左转、右转、急停），支持鼠标点击与键盘事件监听（方向键 + 空格键）。
 - 内置按键防抖与状态反馈机制，确保控制指令的准确下发。
- 2. **实时遥测仪表盘**：
 - 以数据表格形式高频刷新小乌龟的位姿信息：坐标 (x,y) 、航向角 θ 、线速度 v 、角速度 ω 。
 - 显示当前有限状态机（FSM）的状态标签（如 MOVE、TURNING），便于调试与逻辑验证。
- 3. **迷宫可视化画布**：
 - 右侧主区域使用 HTML5 Canvas 绘制 900×620 像素的矢量迷宫地图。
 - 实时渲染小乌龟的当前位置、朝向箭头及历史探索轨迹（黄色轨迹线），直观展示路径规划效果。
- 4. **系统日志与连接管理**：
 - 底部日志面板实时滚动输出后端推送的关键事件（如：初始化完成、▲ 检测到墙壁、🎯 目标接近）。
 - 顶部状态指示灯显示 WebSocket 连接状态（ws://localhost:8765），支持断线重连提示。

三、迷宫定义与算法设计

迷宫核心逻辑封装于 maze.py，采用面向对象思想与模块化设计，主要包含以下技术细节：

1. 栅格化地图数据结构:

- 采用 11×11 的二维列表 (Matrix) 定义拓扑结构, 其中 0 代表可通行区域, 1 代表障碍物/墙壁。
- 每个栅格单元映射 Turtlesim 坐标系中的 1.0 个单位长度, 确保逻辑坐标与物理坐标的 1:1 对应。

2. 双向坐标映射机制:

- **矩阵转物理:** `maze_to_turtle(row, col)` 函数将矩阵索引转换为 Turtlesim 中心坐标, 公式为: $x = 0.5 + col \times 1.0 + 0.5$ $y = 0.5 + (10 - row) \times 1.0 + 0.5$
- **物理转矩阵:** `turtle_to_maze(x, y)` 函数通过逆向运算, 将浮点型物理坐标量化为整数栅格索引, 用于碰撞检测与目标判定。

3. 图搜索与邻居检索:

- `get_neighbors(row, col)` 函数实现了四方向 (上、下、左、右) 的邻居节点检索。
- 内置边界检查与墙壁碰撞判定 `is_wall()`, 为后续扩展 A* 或 Dijkstra 等路径规划算法提供底层接口。

四、桥接逻辑与 WebSocket 通信机制

`turtlesim_web_bridge.py` 作为系统的核心中枢, 采用 **多线程 + 异步 IO** 架构, 融合了 ROS2 节点与 WebSocket 服务端:

1. ROS2 基础通信栈:

- **订阅:** 监听 `/turtle1/pose` 话题, 获取 `geometry_msgs/Pose` 消息, 更新内部状态变量。
- **发布:** 向 `/turtle1/cmd_vel` 话题发布 `geometry_msgs/Twist` 消息, 控制机器人线速度 v_x 与角速度 ω_z 。

2. 有限状态机 (FSM) 控制算法:

- 设计了 MOVE (直线巡航) 与 TURNING (原地转向) 两种核心状态。
- **防碰撞逻辑:** 实时计算机器人到四周边界 ($x_{min}, x_{max}, y_{min}, y_{max}$) 的欧氏距离。当 $d_{min} < 1.5m$ 时, 触发状态切换, 随机选择 $\pm 90^\circ$ 转向, 并在角度误差 $|\Delta\theta| < 0.1 \text{ rad}$ 时恢复前进。

3. 异步通信与并发架构:

- **ROS2 线程:** 在守护线程 (Daemon Thread) 中运行 `rclpy.spin()`, 确保消息回调与定时器任务的实时性。
- **Asyncio 主循环:** 主线程运行 `asyncio` 事件循环, 启动 WebSocket 服务 (端口 8765), 处理前端 JSON 指令并推送状态数据。
- **线程安全:** 通过共享内存变量与适当的锁机制, 解决了 ROS2 回调线程与 Asyncio 主线程之间的数据竞争问题, 实现了低延迟、高并发的跨平台控制链路。